

Università degli Studi di Napoli

Federico II

Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione

Corso di Laurea in Ingegneria dell'Automazione e Robotica

Foundation of Robotics

Analysis and Control of a S.C.A.R.A. Manipulator

Giacomo Caiazzo P38000236

Marco Bartone P38000237

Abstract

This technical project presents a comprehensive analysis of the SCARA (Selective Compliance Assembly Robot Arm) manipulator, covering its kinematic and dynamic modeling, operational space trajectory planning, and motion control. The document details the derivation of the direct kinematics and Jacobian matrix, investigates the manipulator's workspace and dexterity through manipulability analysis, and outlines the implementation of a complex 3D trajectory. A comparative study of various Closed-Loop Inverse Kinematics (CLIK) algorithms is performed to generate the necessary joint-space references. Finally, three distinct motion control strategies – Robust Control, Adaptive Control, and Inverse Dynamics Control in the Operational Space – are implemented and evaluated based on their tracking performance and robustness properties.

Table of Contents

1 Kinematic analysis of the S.C.A.R.A. manipulator **6**

1.1 Frames Definition and Direct Kinematics Computation 6

1.2 Jacobian Computation 8

2 Manipulability analysis and ellipsoids **9**

3 Trajectory planning **14**

3.1 Path primitives definition 16

4 Inverse kinematics algorithms (CLIK) **18**

4.1 Jacobian Inverse CLIK 19

4.2 CLIK with Jacobian Transpose 22

4.3 Jacobian Pseudo-Inverse CLIK for redundancy exploitation 24

4.4 Second order CLIK algorithm 26

5 Motion Control Techniques and Dynamic Modeling **30**

5.1 Joint-Space Dynamic Model Formulation 30

5.1.1 Dynamic model analysis 31

5.2 Dynamic parameters 32

5.3 Robust Control 32

5.3.1 Sliding Mode Control and Chattering Mitigation 33

5.4 Adaptive Control 36

5.5 Inverse Dynamic Control in Operational Space 38

List of Figures

1 S.C.A.R.A. frames 6

2 Manipulability ellipsoids with $q = q_1$ 12

3 Manipulability ellipsoids with $q = q_2$ 12

4 Manipulability ellipsoids with $q = q_3$ 12

5 Manipulability ellipsoids with $q = q_4$ 12

6 S.C.A.R.A. traj 3D 15

7 S.C.A.R.A. traj 2D 15

8 Time evolution of s,$\dot{s}$,$\ddot{s}$ 16

9 Desired ee position 17

10 Desired ee orientation 17

11 Desired ee velocity 18

12 Desired ee acceleration 18

13 Jacobian inverse CLIK algorithm 20

14 End-effector position 20

15 End-effector orientation 20

16 Position tracking error using jacobian inverse 21

17 Joint variables values 21

18 End-effector position 22

19 End-effector orientation 22

20 Position tracking error using jacobian transpose 23

21 Joint variables values 23

22 End-effector position 25

23 End-effector orientation 25

24 End-effector position 25

25 End-effector orientation 25

26 Second order CLIK algorithm Simulink scheme 27

27 Second order CLIK joint variables values 27

28 Second order CLIK joint variables velocities 28

29 Second order CLIK joint variables accelerations 28

30 Second order CLIK position tracking error 29

31 Robust control Simulink scheme 34

32 Joint torques 1-2-4 with $\epsilon = 0.01$ 34

33 Joint 3 torque with $\epsilon = 0.01$ 34

34 Position tracking error with $\epsilon = 0.01$ of joints 1-2-4 35

35 Position tracking error with $\epsilon = 0.01$ of joint 3 35

36 Joint torques 1-2-4 with $\epsilon = 0.3$ 35

37 Joint 3 torque with $\epsilon = 0.3$ 35

38 Position tracking error of joints 1-2-4 with $\epsilon = 0.3$ 36

39 Position tracking error of joint 3 with $\epsilon = 0.3$ 36

40 Adaptive control Simulink scheme 37

41 Joint 1-2-4 torques with adaptive control 38

42 Joint 1-2-4 torques with adaptive control 38

43 Position tracking error of joints 1-2-4 38

44 Position tracking error of joint 3 38

45 Operational space inverse dynamics control Simulink scheme 40

46 Joints 1-2-4 torques 40

47 Joint 3 torque 40

48 Position tracking error of joints 1-2-4 41

49 Position tracking error of joint 3 41

Introduction

The SCARA manipulator is a four-degree-of-freedom (4-DOF) robot, widely used for high-speed, high-precision assembly operations due to its structural compliance in the vertical (z) axis and stiffness in the horizontal (x-y) plane. The typical architecture of a SCARA manipulator consists of the following four degrees of freedom:

- **Two planar revolute joints**, enabling motion within the (x-y) plane.
- **One prismatic joint**, which enables vertical translation along the z-axis.
- **One final revolute joint**, dedicated to the orientation of the end-effector about its own axis.

In the S.C.A.R.A. manipulator, although actuation is typically achieved through electric motors, the kinematic structure exhibits distinct characteristics. The coupling between joint and Cartesian variables is direct mainly along the vertical (z) axis, while in the horizontal plane the motion is highly coupled and nonlinear. Moreover, due to its serial kinematic chain, the end-effector's positioning accuracy progressively degrades with the radial distance from the base.

The forthcoming section will provide a detailed analysis of the manipulator's kinematic model, elucidating the mathematical equations that define the relationship between the joint variables and the end-effector pose.

1 Kinematic analysis of the S.C.A.R.A. manipulator

The fundamental objective of direct kinematics lies in determining the precise **position and orientation** (pose) of the manipulator's end-effector frame relative to the base frame. Formally, this relationship is expressed through the kinematic function $\boldsymbol{k} : \mathbb{R}^m \to \mathbb{R}^n$, which maps the set of $\boldsymbol{n}$ joint variables, represented by the vector $\boldsymbol{q}$, to the $\boldsymbol{m}$-dimensional vector of operational space variables, $\boldsymbol{x_e}$. This correlation is concisely stated as:

$$x_e = k(q) \tag{1}$$

where $x_\mathbf{e}$ encapsulates the end-effector's spatial configuration, and $q = [\theta_1 \quad \theta_2 \quad d_3 \quad \theta_4]^T$ is the vector of joint coordinates, where $\theta_1, \theta_2, \theta_4$ are angular displacements for the revolute joints and d_3 is the linear displacement for the prismatic joint.

1.1 Frames Definition and Direct Kinematics Computation

As an initial procedural step in the computation of direct kinematics, the right-handed frames are systematically affixed to each link, rigorously adhering to the Denavit-Hartenberg (DH) convention.

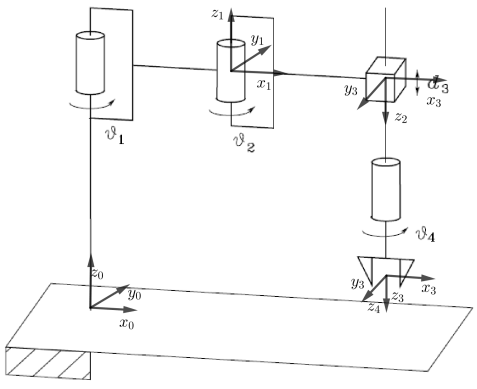


Figure 1: S.C.A.R.A. frames

This systematic framing facilitates the derivation of the homogeneous transformation matrix, $\boldsymbol{T}_4^0(\mathbf{q})$, which defines the position and orientation of the end-effector frame (Frame 4) relative to the fixed base frame (Frame 0):

$$T_4^0(q) = A_1^0(\theta_1)A_2^1(\theta_2)A_3^2(d_3)A_4^3(\theta_4) \tag{2}$$

The generic homogeneous transformation matrix, $\boldsymbol{A}_i^{i-1}$, relating frame i to frame $i-1$ is formally defined by the four DH parameters $(\theta_i, \alpha_i, a_i, d_i)$:

$$A_i^{i-1} = \begin{pmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{3}$$

For the SCARA manipulator under consideration, the respective DH parameters are summarized in Table 1

Link	$\boldsymbol{a_i}$	$\boldsymbol{\alpha_i}$	$\boldsymbol{d_i}$	$\boldsymbol{\theta_i}$
1	0.5	0	1	θ_1
2	0.5	π	0	θ_2
3	0	0	d_3	0
4	0	0	0	θ_4

Table 1: Denavit-Hartenberg parameters for the SCARA manipulator.

Notwithstanding the utility of the DH formalism, an analytical approach utilizing homogeneous transformation matrices often introduces considerable computational complexity. Consequently, a more efficient alternative is to employ the **geometric method**, which capitalizes on the specific and planar simplicity inherent to the SCARA manipulator's architecture.

By employing the geometric resolution method, the end-effector's pose, represented by the vector of operational space variables $\mathbf{x_e}$, is directly formulated as a function of the joint variables vector $\mathbf{q}$:

$$x_e = k(q) = \begin{pmatrix} x \\ y \\ z \\ \phi \end{pmatrix} = \begin{pmatrix} a_1c_1 + a_2c_{12} \\ a_1s_1 + a_2s_{12} \\ d_0 - d_3 \\ \theta_1 + \theta_2 - \theta_4 \end{pmatrix} \tag{4}$$

1.2 Jacobian Computation

In this subsection, the focus is placed on deriving the *Jacobian matrix* for the SCARA manipulator. The Jacobian plays a pivotal role in the kinematic analysis, as it establishes the differential relationship between the joint velocities and the linear and angular velocities of the end-effector:

$$v_e = J(q)\dot{q} \tag{5}$$

The geometric jacobian is typically partitioned into the (3x1) columns vectors J_{pi} and J_{oi}:

$$J(q) = \begin{pmatrix} J_{p_1} & \cdots & J_{p_N} \\ J_{o_1} & \cdots & J_{o_N} \end{pmatrix} \tag{6}$$

where:

- J_{p_i} quantifies the contribution of the i-th joint to the **linear velocity** of the end-effector.
- J_{o_i} represents the contribution of the i-th joint to the **angular velocity** of the end-effector.

The structure of each i-th column $\begin{pmatrix} J_{p_i} \\ J_{o_i} \end{pmatrix}$ is dictated by the specific type of joint (revolute or prismatic) i:

$$\begin{pmatrix} J_{p_i} \\ J_{o_i} \end{pmatrix} = \begin{cases} \begin{pmatrix} \mathbf{z}_{i-1} \\ \mathbf{0} \end{pmatrix} & \text{for a prismatic joint} \\ \begin{pmatrix} \mathbf{z}_{i-1} \times (p_e - p_{i-1}) \\ \mathbf{z}_{i-1} \end{pmatrix} & \text{for a revolute joint} \end{cases} \tag{7}$$

The form the Jacobian matrix assumes for the S.C.A.R.A. manipulator is the following:

$$J(q) = \begin{bmatrix} z_0 \times (p_e - p_0) & z_1 \times (p_e - p_1) & z_2 & z_3 \times (p_e - p_3) \\ z_0 & z_1 & 0 & z_3 \end{bmatrix}, \tag{8}$$

where z_i denotes the unit vector along the z-axis of the i-th reference frame, and p_i is the position vector of the origin of frame $\{i\}$ with respect to the base frame $\{0\}$. The vector p_e represents the position of the end-effector.

Exploiting the structural simplicity of the SCARA manipulator and substituting the kinematic relationships obtained from the direct kinematics, the explicit expression of the Jacobian matrix

becomes:

$$J(q) = \begin{bmatrix} -a_1 s_1 - a_2 s_{12} & -a_2 s_{12} & 0 & 0 \\ a_1 c_1 + a_2 c_{12} & a_2 c_{12} & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & -1 \end{bmatrix} \tag{9}$$

It can be observed from Equation (9) that rotations about the x and y axes are not feasible for this manipulator configuration. Consequently, when restricting the analysis to a four-dimensional operational space, the analytical Jacobian can be obtained by removing the fourth and fifth rows of the geometric Jacobian, yielding:

$$J_A(q) = \frac{\partial k(q)}{\partial q} = \begin{bmatrix} -a_1 s_1 - a_2 s_{12} & -a_2 s_{12} & 0 & 0 \\ a_1 c_1 + a_2 c_{12} & a_2 c_{12} & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{10}$$

The obtained analytical Jacobian $J_A(q)$ will serve as the fundamental mapping between the joint velocities and the operational space velocities, and it will be extensively employed in the subsequent analysis concerning the inverse kinematics and control algorithms.

2 Manipulability analysis and ellipsoids

This section is dedicated to the **manipulability analysis** of the SCARA robot's supporting structure. Manipulability serves as a measure of the manipulator's intrinsic capacity to execute changes in the end-effector's position as a function of the instantaneous joint configuration. Specifically, a larger manipulability index correlates directly with a greater range of instantaneous motions achievable at that configuration. Conversely, the manipulability measure diminishes to zero when the manipulator reaches a **singular configuration**, where it loses the ability to develop forces or velocities along at least one direction.

To quantitatively assess the end-effector's competence in generating velocities and forces in any specified direction for a given configuration, two principal tools are employed: the **velocity manipulability ellipsoid** and the **force manipulability ellipsoid**. These geometrical constructs are mathematically defined by the following quadratic forms:

- **Velocity Manipulability Ellipsoid**:

$$v_e^T \left(J(q)J(q)^T\right)^{-1} v_e = 1 \tag{4}$$

- **Force Manipulability Ellipsoid**:

$$\gamma_e^T \left(J(q)J(q)^T\right) \gamma_e = 1 \tag{5}$$

In Equations (4) and (5), $J(q)$ denotes the geometric Jacobian matrix, while v_e and γ_e represent the vectors of end-effector velocities and end-effector forces, respectively.

A noteworthy property arises from the inverse proportionality between the matrices at the core of these quadratic forms, which highlights the principle of **kineto-static duality**. This duality dictates that the directions of the principal axes of the velocity ellipsoid coincide with those of the force ellipsoid. However, their physical dimensions are in inverse proportion, illustrating the inherent trade-off between the ability to generate motion (v_e) and the capacity to exert forces (γ_e).

For the SCARA manipulator, the functional core is its supporting structure, which is kinematically analogous to a simple planar two-link arm. Since all the **z**-axes are parallel, the analysis of these ellipsoids is confined exclusively to the **x-y** plane. Consequently, the relevant kinematic model is sufficiently described by the following reduced 2×2 Jacobian:

$$J(q) = \begin{pmatrix} -a_1 s_1 - a_2 s_{12} & -a_2 s_{12} \\ a_1 c_1 + a_2 c_{12} & a_2 c_{12} \end{pmatrix} \tag{6}$$

To obtain a globally representative and scalar quantification of the manipulator's dexterity, we introduce the **manipulability measure**, $w(q)$. This measure is fundamentally derived from the volume of the manipulability ellipsoid. Its general expression, applicable to both redundant and non-redundant systems, is given by:

$$w(q) = \sqrt{\det\left(J(q)J^T(q)\right)} \tag{11}$$

For the specific case under examination, where the supporting structure is considered non-redundant (i.e., the number of operational space degrees of freedom n equals the number of joint degrees of freedom m), the expression simplifies significantly to the absolute value of the determinant of the Jacobian matrix:

$$w(q) = \left|\det\left(J(q)\right)\right| \tag{12}$$

This scalar metric consistently retains a positive value, except when the manipulator occupies a **singular configuration**, where $w(q)$ reduces to zero. Consequently, $w(q)$ serves as an effective distance metric from such singular points in the joint space.

Applying this non-redundant formulation to the reduced 2×2 Jacobian of the SCARA's planar structure (governed solely by θ_1 and θ_2), the manipulability measure is explicitly defined by:

$$w(q) = |a_1 a_2 \sin(\theta_2)| \tag{13}$$

As the equation 13 reveals, the manipulator reaches a singular configuration when $\theta_2 = 0$ or $\theta_2 = \pi$, since the determinant of $J(q)$ vanishes when the second revolute joint attains one of these values.

It is evident from equation 13 that the dexterity of the SCARA arm within the **x**-**y** plane is influenced exclusively by the second joint angle, θ_2. To explore the sensitivity of $w(q)$, various high-dexterity configurations were selected for subsequent analysis, focusing specifically on variations in this critical joint variable:

$$q_1 = \begin{pmatrix} \pi/4 \\ -\pi/2 \end{pmatrix}, \quad q_2 = \begin{pmatrix} \pi/4 \\ -\pi/3 \end{pmatrix}, \quad q_3 = \begin{pmatrix} \pi/4 \\ -\pi/6 \end{pmatrix}, \quad q_4 = \begin{pmatrix} \pi/4 \\ -\pi/9 \end{pmatrix} \tag{14}$$

The results of the simulation are the following:

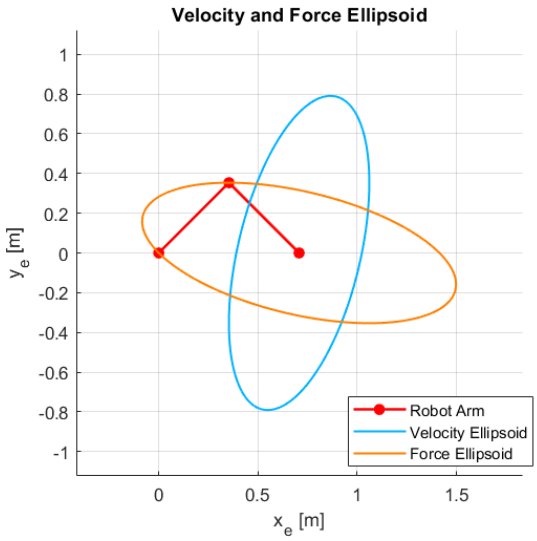


Figure 2: Manipulability ellipsoids with $q = q_1$

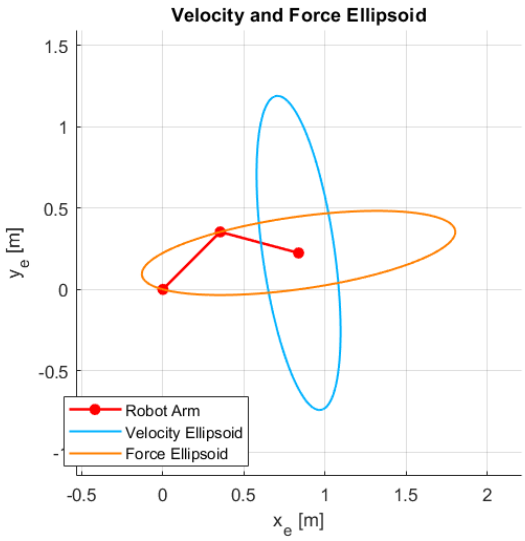


Figure 3: Manipulability ellipsoids with $q = q_2$

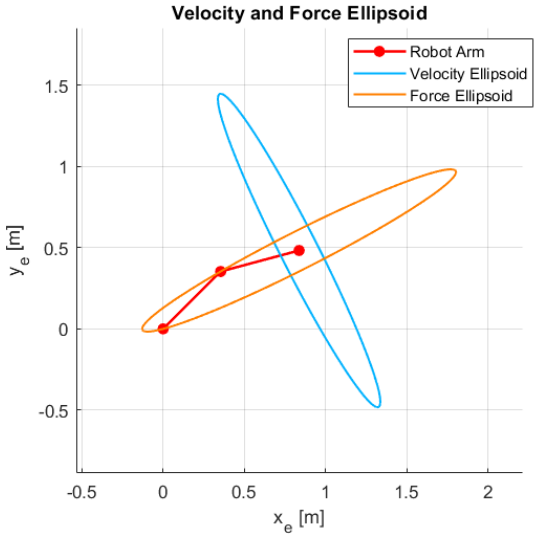


Figure 4: Manipulability ellipsoids with $q = q_3$

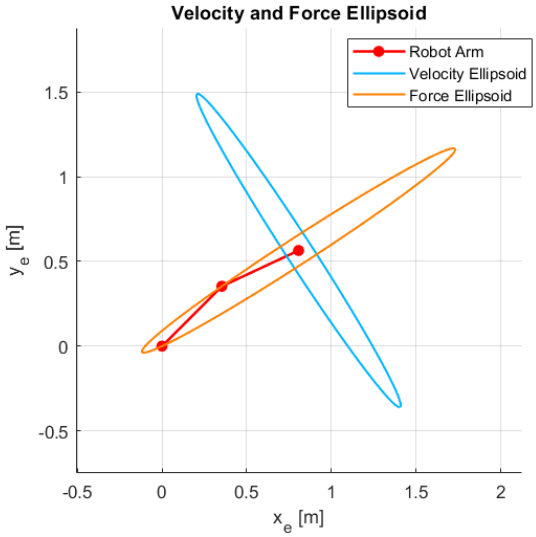


Figure 5: Manipulability ellipsoids with $q = q_4$

The evaluation of the manipulator dexterity for the selected joint configurations yields the following quantitative results for the manipulability measure, $w(q)$:

$$w(q_1) = 0.2500, \quad w(q_2) = 0.2165, \quad w(q_3) = 0.125, \quad w(q_4) = 0.0855$$

As anticipated by the derived analytical expression, the condition of maximum dexterity is attained for a value of the second joint variable equal to $\frac{\pi}{2}$. Conversely, the manipulability exhibits a proportional decrease, signifying a loss of dexterity, as the manipulator's elbow moves towards a stretched or

retracted configuration (i.e., when θ_2 approaches 0 or π radians). This correlation directly confirms that the manipulability measure diminishes precisely as the kinematic configuration approaches a singular configuration.

3 Trajectory planning

In this section, we focus on the trajectory planning for the SCARA manipulator. The main aim is to generate a path consisting of a fixed number of points, in such a way to incorporate both straight and circular segments, using a limited number of *via points*, namely, specific points in the operational space, that a manipulator should pass near, rather than exactly through, to ensure smooth and continuous movement. In the following, the list of points that were chosen for the trajectory is shown (the ones in table were rescaled of a factor of 2.5):

Point	**X [m]**	**Y [m]**	**Z [m]**	ϕ **[rad]**
P_1	0.60	0.30	0.30	0
P_2	0.90	0.35	0.45	0
P_3	1.00	0.50	0.60	0
P_4	1.20	0.70	0.90	$\pi/4$
P_5	1.25	0.95	1.00	$\pi/6$
P_6	1.00	0.80	0.85	$\pi/6$
P_7	0.90	0.75	0.60	$\pi/6$
P_8	0.60	0.90	0.90	$\pi/6$
P_9	0.45	0.70	0.90	$\pi/6$
P_{10}	0.35	0.88	0.65	$\pi/6$
P_{11}	0.25	0.88	0.50	$\pi/6$
P_{12}	0.30	0.90	0.50	$\pi/6$
P_{13}	0.00	0.90	0.50	$\pi/6$
P_{14}	-0.20	0.90	0.50	$\pi/4$
P_{15}	-0.35	0.78	0.70	$\pi/4$
P_{16}	-0.35	0.60	0.70	$\pi/4$
P_{17}	-0.65	0.40	0.55	$\pi/4$
P_{18}	-0.80	0.35	0.35	$\pi/4$
P_{19}	-0.40	0.25	0.15	$\pi/4$
P_{20}	0.00	0.30	0.25	$\pi/4$
P_{21}	0.20	0.45	0.35	$\pi/4$
P_{22}	0.45	0.60	0.40	$\pi/6$
P_{23}	0.60	0.30	0.30	0

Table 2: Selected Trajectory Points in Operational Space.

The trajectory in the operational space is the following:

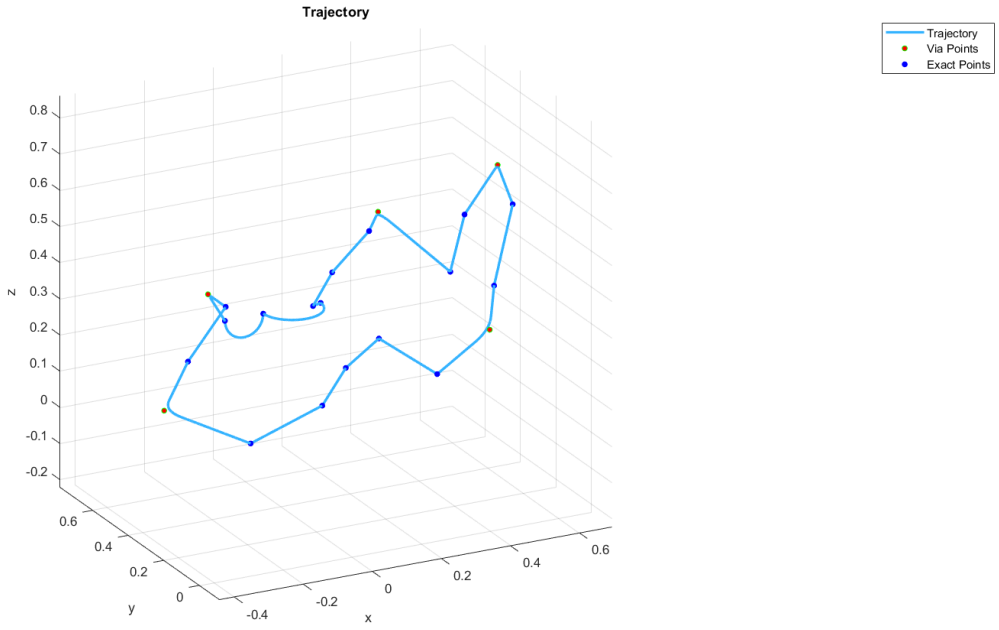


Figure 6: S.C.A.R.A. traj 3D

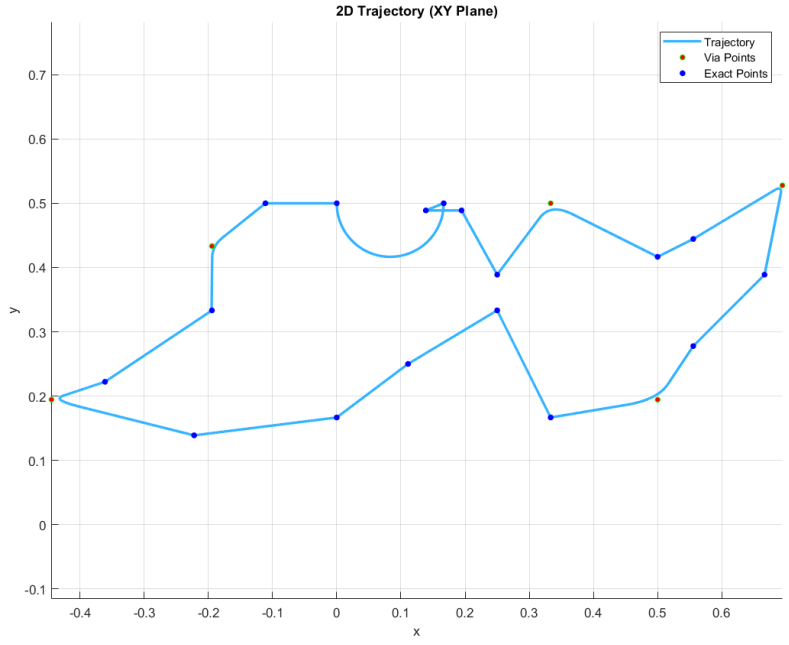


Figure 7: S.C.A.R.A. traj 2D

3.1 Path primitives definition

Once the requisite path points are established, the ensuing objective is to define the specific motion primitives that govern the end-effector's transition between consecutive poses. This necessitates the provision of a rigorous, mathematically defined law specifying the movement from one spatial configuration to the next.

To accurately characterize these laws of motion along the defined geometric primitives, a parametric representation of the curve in space is necessary. This representation is achieved through the utilization of a curvilinear abscissa, s, where specific discrete values, s_k, are assigned to each path point p_k, satisfying the condition:

$$s_k : p(s_k) = p_k$$

This parametric formulation allows the temporal behavior of the trajectory to be decoupled from its spatial geometry. The final step required to fully define the end-effector trajectory involves establishing a **timing law** that dictates how the curvilinear abscissa, s, evolves over time. This temporal law defines the motion profile along the predefined path primitives. In the present analysis, a series of trapezoidal velocity profiles were chosen to govern the time evolution of the curvilinear abscissa.

The time evolution profiles for the curvilinear abscissa, $s(t)$, its derivative, $\dot{s}(t)$, and its second derivative, $\ddot{s}(t)$, are presented hereafter for the specified operational time interval of $[0, 120]$ seconds:

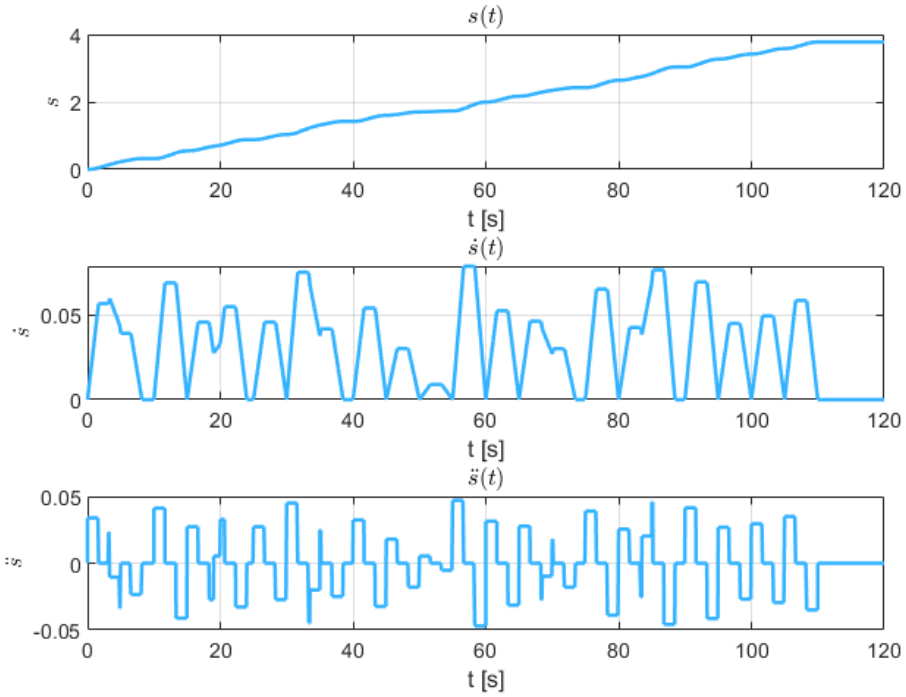


Figure 8: Time evolution of s,$\dot{s}$,$\ddot{s}$

The fundamental functions describing the geometric path, $P(s)$, along with the established temporal law for the curvilinear abscissa, $s(t)$, permit the explicit derivation of the time-dependent end-effector trajectory, $P(t)$, and its corresponding derivatives:

- **Position**: The trajectory $P(t)$ is directly obtained by substituting the temporal law into the spatial path function:
$$P(t) = P(s(t)) \tag{15}$$
- **Velocity**: The end-effector velocity, $\dot{P}(t)$, is computed by applying the chain rule, where $P'(s) = \frac{dP(t)}{ds}$ represents the tangent vector along the path:
$$\dot{P}(t) = \frac{dP(t)}{ds}\dot{s}(t) \tag{16}$$
- **Acceleration**: The end-effector acceleration, $\ddot{P}(t)$, is found by differentiating the velocity expression $\dot{P}(t)$ with respect to time, which results in:
$$\ddot{P}(t) = \frac{d\dot{P}(t)}{ds}\dot{s}(t) \tag{17}$$

Hereafter, the time evolution profiles for the four components of the end-effector's desired pose vector, $x_d(t)$, along with its respective first $\dot{x}_d(t)$ and second $\ddot{x}_d(t)$ time derivatives, are graphically represented:



Figure 9: Desired ee position

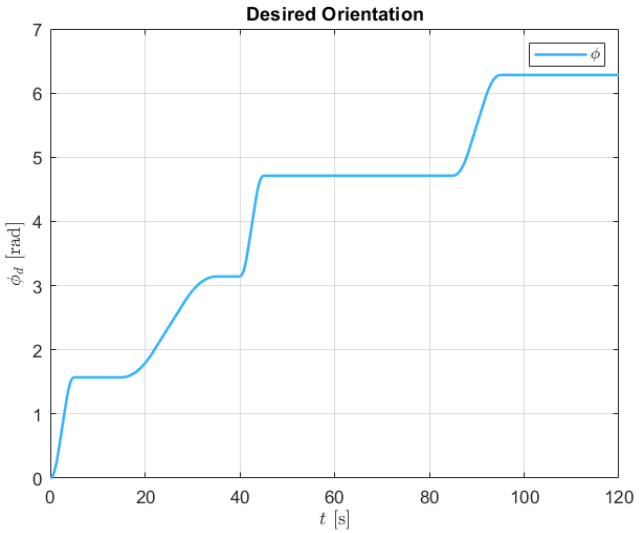


Figure 10: Desired ee orientation

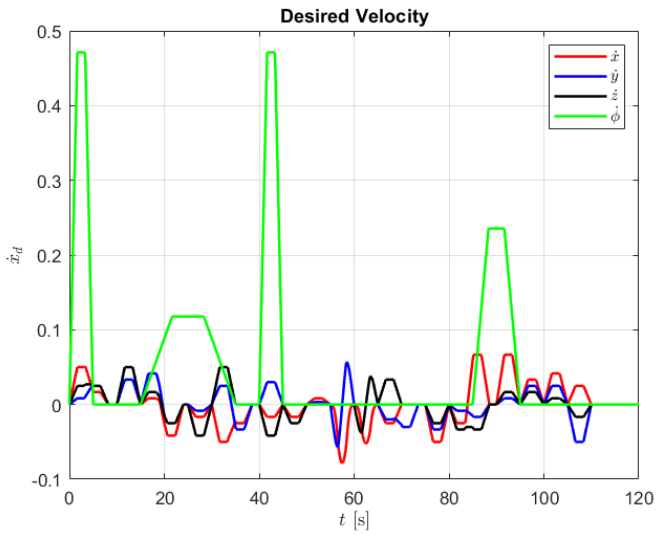


Figure 11: Desired ee velocity

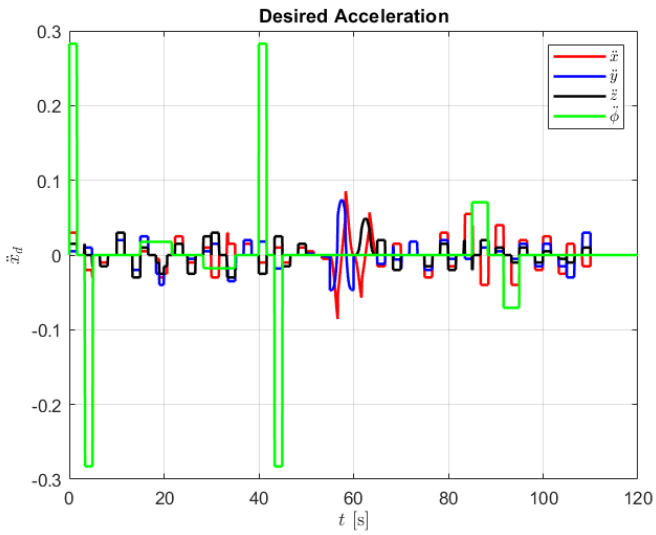


Figure 12: Desired ee acceleration

4 Inverse kinematics algorithms (CLIK)

Given the desired end-effector pose, velocity, and acceleration defined in the operational space, the primary objective is to determine the corresponding joint positions, velocities, and accelerations within the joint space. This crucial transformation is achieved through **Closed-Loop Inverse Kinematics (CLIK)** algorithms, which incorporate feedback control principles to ensure accurate tracking. The foundation of these methods lies in defining a **tracking error** in the operational space:

$$e = x_d - x_e \tag{18}$$

where x_d and x_e represent the vectors of the desired and the computed end-effector pose, respectively.

Differentiating the tracking error (Equation 18) with respect to time yields the error dynamics:

$$\dot{e} = \dot{x}_d - \dot{x}_e$$

By substituting the fundamental differential kinematics relationship, $\dot{x}_e = J_A(q)\dot{q}$, into the error dynamics expression, we obtain:

$$\dot{e} = \dot{x}_d - J_A(q)\dot{q} \tag{19}$$

The underlying principle of the ensuing CLIK algorithms is to express the joint velocity vector, $\dot{q}$, as a dedicated function of the tracking error, e. This formulation establishes a closed-loop differential equation whose solution dictates the required convergence dynamics of the error over time.

The following core CLIK strategies are implemented and analyzed for the SCARA manipulator:

- **Jacobian Inverse CLIK**: A computationally intensive yet highly effective strategy that relies on computing the inverse of the analytical Jacobian, $J_A^{-1}(q)$.
- **Jacobian Transpose CLIK**: A computationally simpler alternative which employs the Jacobian transpose, $J_A^T(q)$. Although less demanding, this method may exhibit a slightly degraded error tracking capability.
- **Jacobian Pseudo-Inverse CLIK**: A modification designed for functionally redundant manipulators, utilizing the Jacobian right pseudo-inverse, $J_A^\dagger(q)$, and exploiting redundant degrees of freedom (DOFs) to maximize a secondary objective function.
- **Second-Order CLIK**: An advanced approach necessary for control schemes that require explicit joint acceleration inputs, as it directly computes the required vector of joint accelerations, $\ddot{q}$.

.

4.1 Jacobian Inverse CLIK

Assuming the Analytical Jacobian matrix, $J_A(q)$, is both square and non-singular, a suitable control law for the joint velocity vector, $\dot{q}$, can be defined using the inverse Jacobian:

$$\dot{q} = J_A^{-1}(q)(\dot{x}_d + Ke) \tag{20}$$

where K is a positive-definite gain matrix. Substituting this control law (Equation 20) into the fundamental error kinematics (Equation 19) yields the following linear, first-order differential equation governing the tracking error dynamics:

$$\dot{e} + Ke = 0 \tag{21}$$

The global asymptotic stability of the closed-loop system is assured by the positive-definiteness of matrix K. Furthermore, the convergence rate, by which the error e tends asymptotically to zero along the system trajectories, is directly determined by the eigenvalues of K. Hereafter, the scheme which implements the aforementioned kinematic algorithm in Simulink is shown:

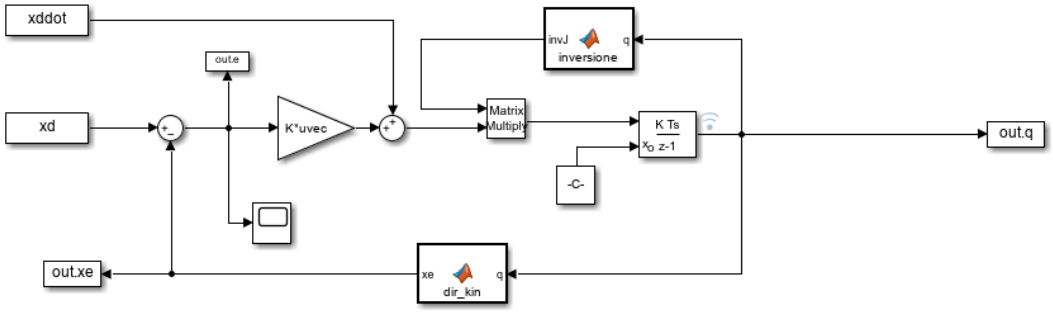


Figure 13: Jacobian inverse CLIK algorithm

The simulation results obtained by running the model over the time interval $[0, 120]$ s demonstrate the resulting error convergence and trajectory tracking performance.

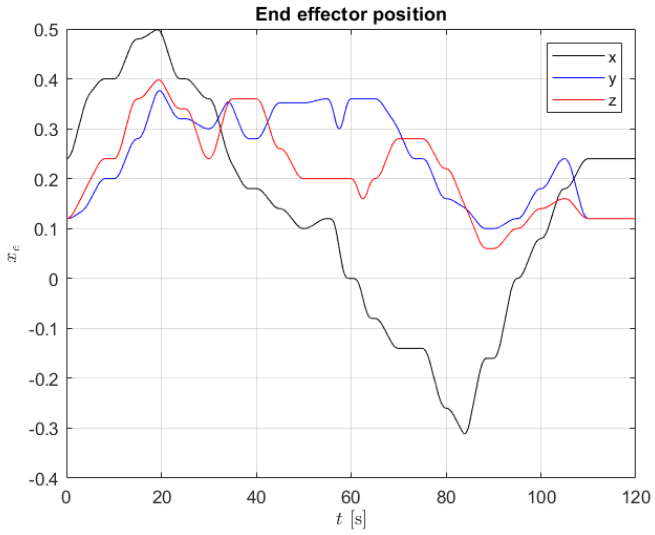


Figure 14: End-effector position

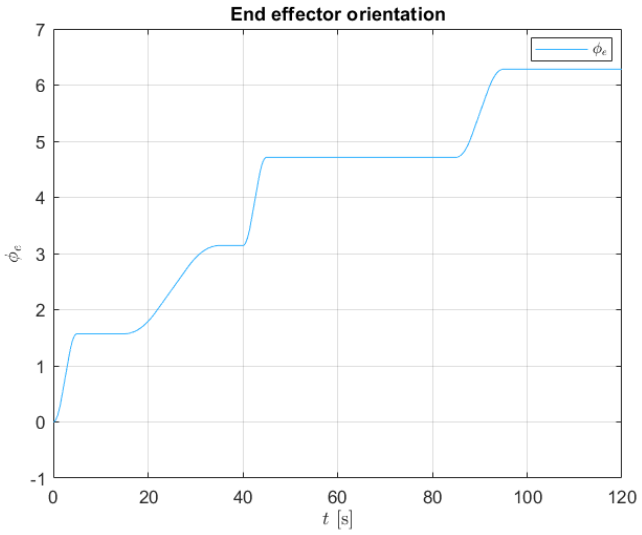


Figure 15: End-effector orientation

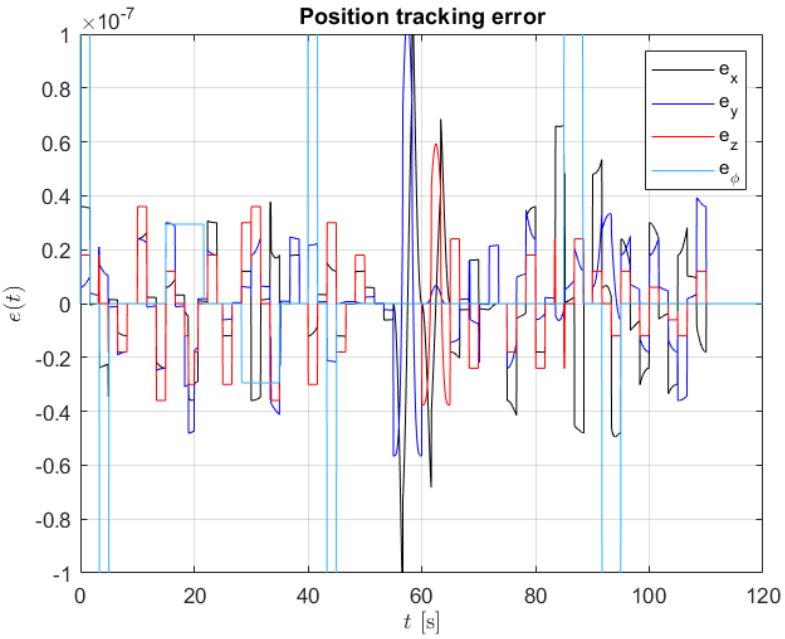


Figure 16: Position tracking error using jacobian inverse

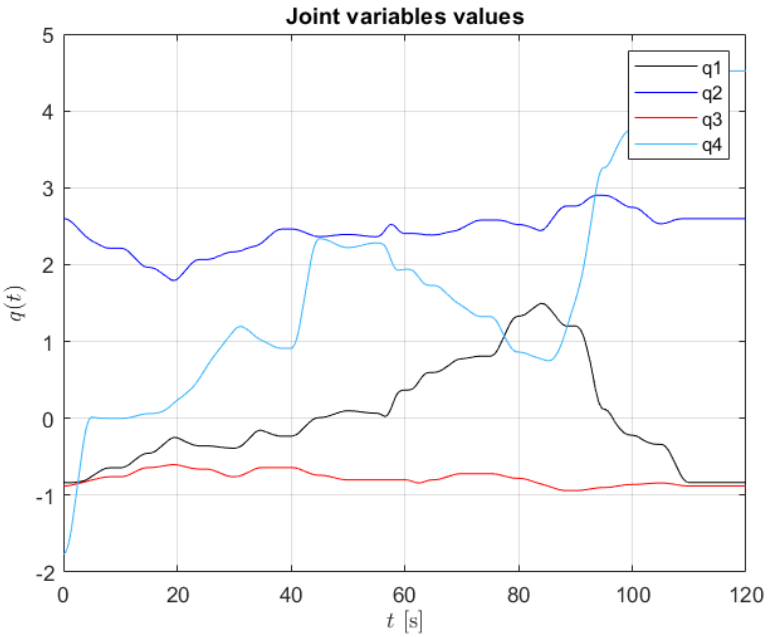


Figure 17: Joint variables values

The primary drawback associated with the Jacobian Inverse algorithm is its inherent lack of robustness near kinematic singularities. In such critical configurations, the Jacobian matrix is non-invertible ($\det(J_A(q)) = 0$). Even when merely operating in the vicinity of these singularities (i.e., when $J_A(q)$ is ill-conditioned), the inversion process mathematically necessitates extremely high joint velocities to produce even modest operational space velocities, potentially leading to instability and actuator saturation.

To mitigate this critical issue, an alternative approach is provided by the **Jacobian Transpose CLIK** algorithm. Since this method explicitly avoids the computationally unstable process of matrix inversion, it offers enhanced robustness for kinematic inversion, particularly when the manipulator configuration approaches singularities.

4.2 CLIK with Jacobian Transpose

A suitable alternative to the Jacobian Inverse method is the **Jacobian Transpose CLIK** algorithm, where the joint velocity, $\dot{q}$, is determined by the following linear relationship:

$$\dot{q} = J_A^T(q)Ke \tag{22}$$

In this formulation, K is a positive-definite gain matrix. This selection for $\dot{q}$ leads to a significantly simpler computational algorithm because it explicitly replaces the computationally demanding and potentially unstable process of matrix inversion with the transpose operation. Consequently, this method inherently **avoids critical issues** related to kinematic and representation singularities, where the Jacobian matrix, $J_A(q)$, may be singular or ill-conditioned.

Simulation results are presented in the following:

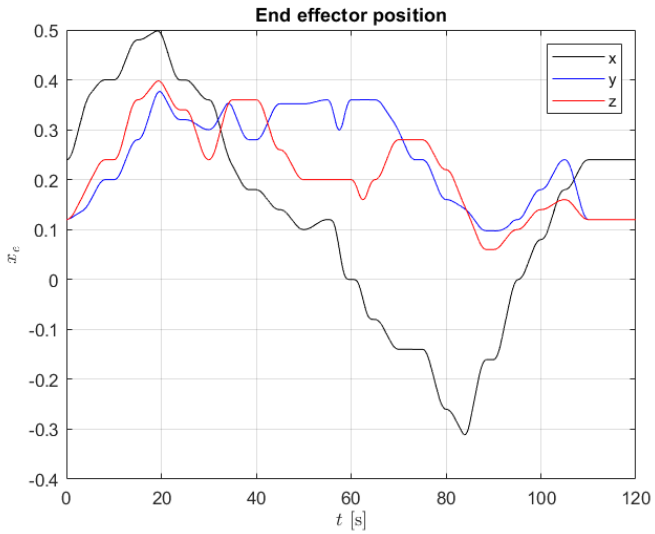


Figure 18: End-effector position

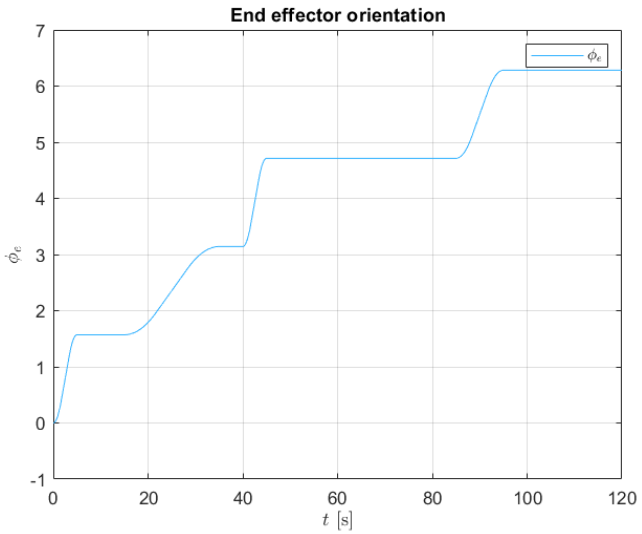


Figure 19: End-effector orientation

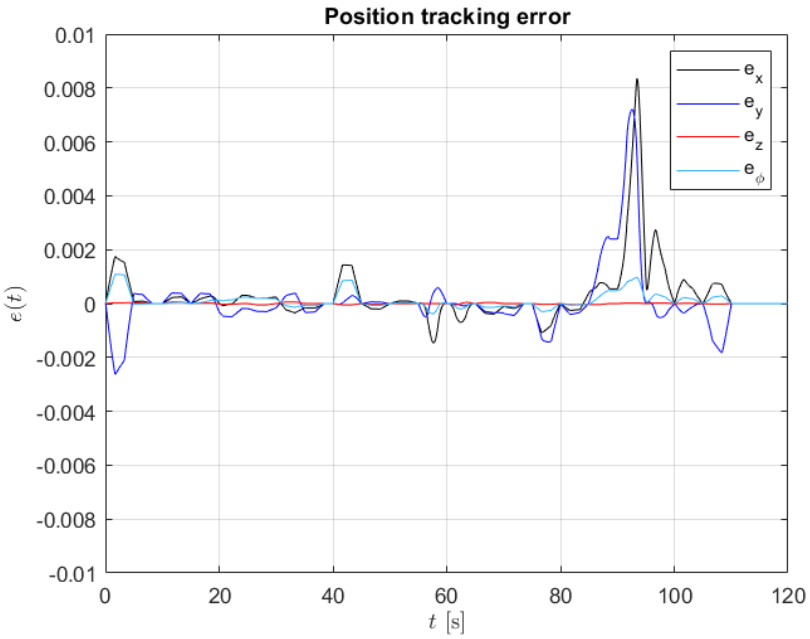


Figure 20: Position tracking error using jacobian transpose

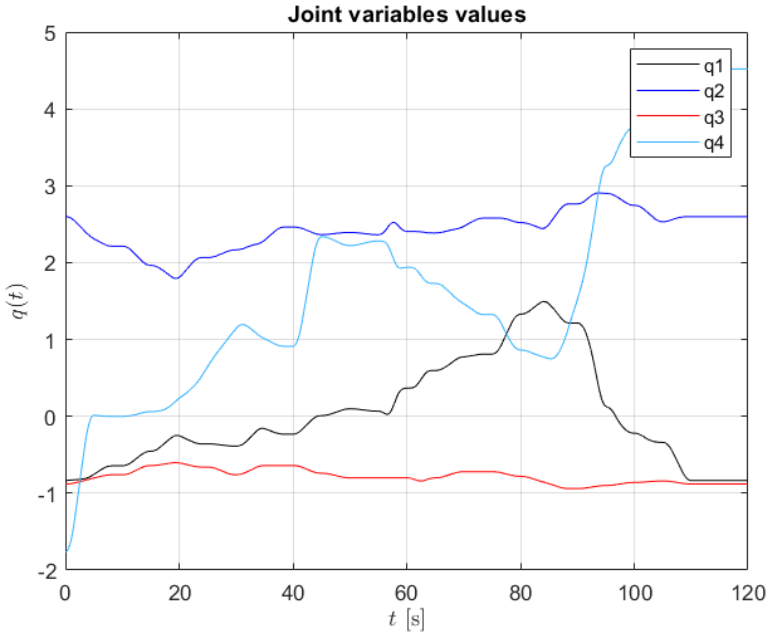


Figure 21: Joint variables values

However, the principal drawback of this approach resides in its potential for **degraded tracking performance**. Specifically, the algorithm may fail to ensure perfect tracking of the operational space error when the desired reference input, x_d, undergoes rapid variations over time. As can be readily observed from the simulated error profile, the magnitude of the tracking error achieved with the Jacobian Transpose method is on the order of 10^{-3}. This constitutes a degradation in accuracy

compared to the preceding case utilizing the Jacobian Inverse algorithm, where the error magnitude remained significantly lower, on the order of 10^{-7}.

4.3 Jacobian Pseudo-Inverse CLIK for redundancy exploitation

When a redundant manipulator is available, kinematic inversion can be effectively performed using the CLIK algorithm with the Jacobian Pseudo-Inverse. This algorithm is crucial because, in addition to generating the joint position references corresponding to the operational space trajectory, it allows for the exploitation of the manipulator's additional degrees of freedom (DOFs) to pursue a secondary objective function.

This objective is achieved by calculating the joint velocities $\dot{q}$ using the following extended expression:

$$\dot{q} = J_A^{\dagger}(\dot{x}_d + Ke) + \left(I_n - J_A^{\dagger} J_A\right) \dot{q}_0 \tag{23}$$

Where the terms are defined as follows:

- $J_A^{\dagger}(q)$ is the right pseudo-inverse of the Analytical Jacobian in the current configuration.
- $\dot{x}_d$ is the desired velocity vector of the end-effector.
- $e = x_d - x_e$ is the position tracking error in the operational space.
- K is a positive-definite gain matrix.

The term $P = \left(I_n - J_A^{\dagger} J_A\right)$ is called the projector into the null space of the Jacobian, and it satisfies the property $\mathcal{N}(J_A) = \mathcal{R}(P)$. Consequently, the component $\left(I_n - J_A^{\dagger} J_A\right) \dot{q}_0$ enables the modification of joint velocities to induce a change in the manipulator's configuration without altering the end-effector's velocity. This mechanism allows the robot to continue tracking the desired trajectory while simultaneously performing an internal motion.

The velocity vector $\dot{q}_0$ is chosen to maximize or minimize a function $w(q)$ in order to pursue the secondary objective, typically defined as:

$$\dot{q}_0 = k_0 \left(\frac{\partial w(q)}{\partial q}\right)^T \tag{24}$$

Where:

- $k_0 > 0$ is a positive scalar gain factor; since $k_0 > 0$ is chosen, the objective function $w(q)$ is being **maximized**.

- $w(q)$ is the function to be optimized.

Since the SCARA manipulator is inherently non-redundant for its full task, the study of this algorithm required the introduction of functional redundancy. This was achieved by relaxing one of the components of the operational space (specifically, the third component), thereby granting the manipulator relative redundancy with respect to the constrained task.

Hereafter, simulations results are presented:

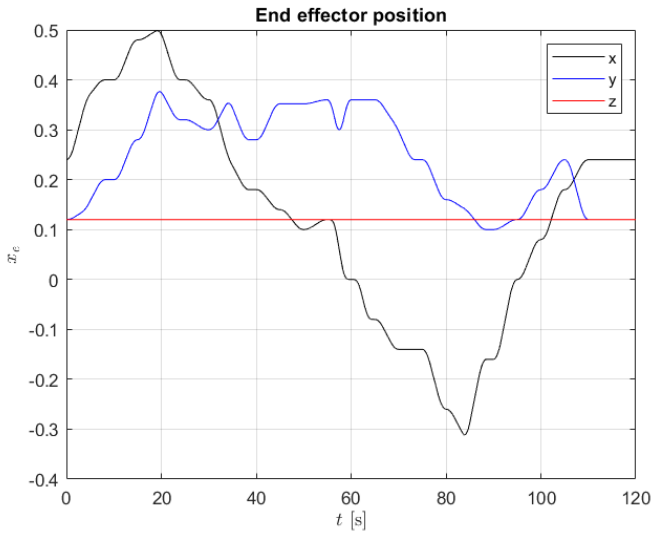


Figure 22: End-effector position

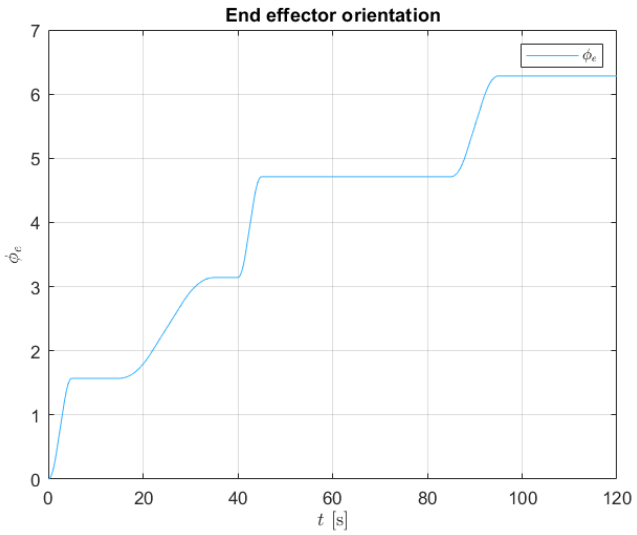


Figure 23: End-effector orientation

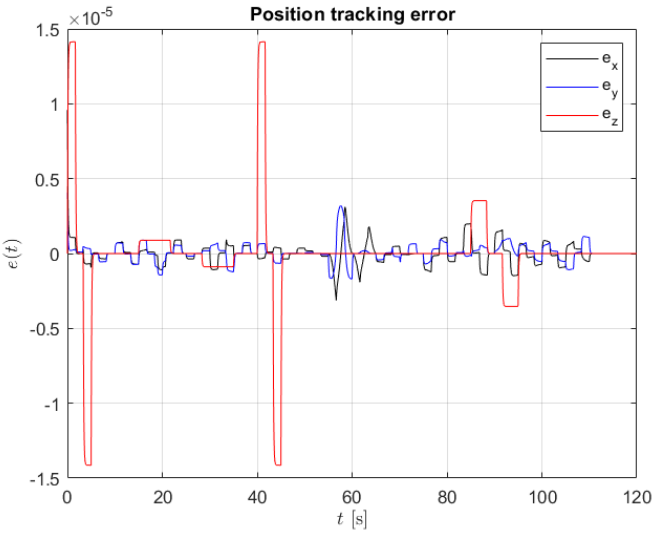


Figure 24: End-effector position

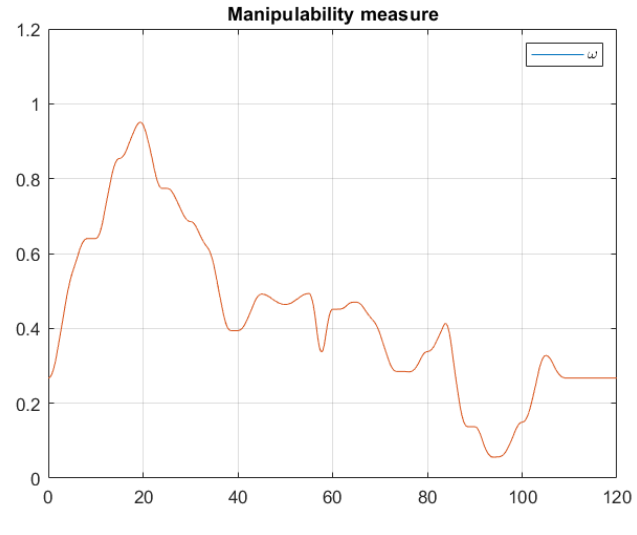


Figure 25: End-effector orientation

The different behaviour in the joint variable trajectories, when compared to the corresponding non-redundant CLIK implementation (e.g., Jacobian Inverse), is directly attributable to the es-

tablishment of functional redundancy through the intentional relaxation of one operational space constraint. In this specific scenario, the kinematic inversion procedure was primarily tasked with accurate tracking of the **x**, **y**, and ϕ coordinates, allowing the **z**-component to be implicitly governed by internal joint motions.

Consequently, the implemented algorithm has successfully computed joint trajectories that satisfy two simultaneous objectives: first, the accurate adherence to the desired reference for the high-priority **x**, **y**, and ϕ coordinates; and second, the concurrent utilization of the Jacobian's null space to maximize the manipulability measure. It is paramount to emphasize that the trajectory tracking task retains the **highest priority** within the overall control hierarchy.

4.4 Second order CLIK algorithm

When the kinematic inversion of a trajectory requires explicit reference generation not only for position and velocity but also for acceleration, it is necessary to employ the second-order CLIK algorithm. This method computes the vector of required joint accelerations, $\ddot{q}$, to enforce the desired motion profile in the operational space. The joint acceleration command is defined as:

$$\ddot{q} = J_A^{-1}(q) \left(\ddot{x}_d + K_D \dot{e} + K_P e - \dot{J}_A(q, \dot{q}) \dot{q} \right) \tag{25}$$

Where the input terms and parameters are rigorously defined:

- $J_A(q)$ is the analytical Jacobian matrix evaluated at the current joint configuration.
- $\ddot{x}_d$ is the desired acceleration vector of the end-effector.
- $e = x_d - x_e$ is the position tracking error.
- $\dot{e} = \dot{x}_d - \dot{x}_e$ is the velocity tracking error.
- K_D and K_P are positive-definite gain matrices (Derivative and Proportional, respectively), utilized to define the asymptotic convergence rate of the error dynamics.
- $\dot{J}_A(q, \dot{q})\dot{q}$ represents the non-linear velocity coupling term arising from the time derivative of the Jacobian matrix.

The algorithm was implemented in Simulink as follows:

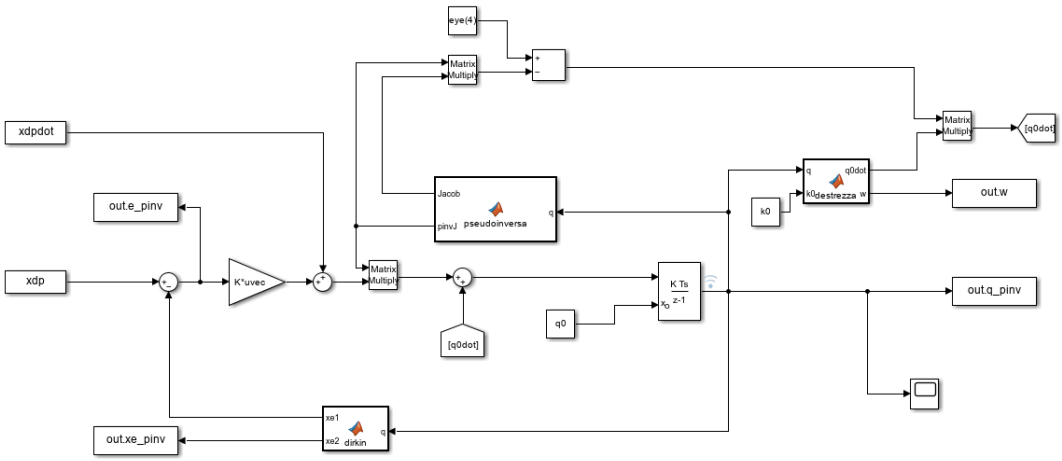


Figure 26: Second order CLIK algorithm Simulink scheme

Simulations results are shown:

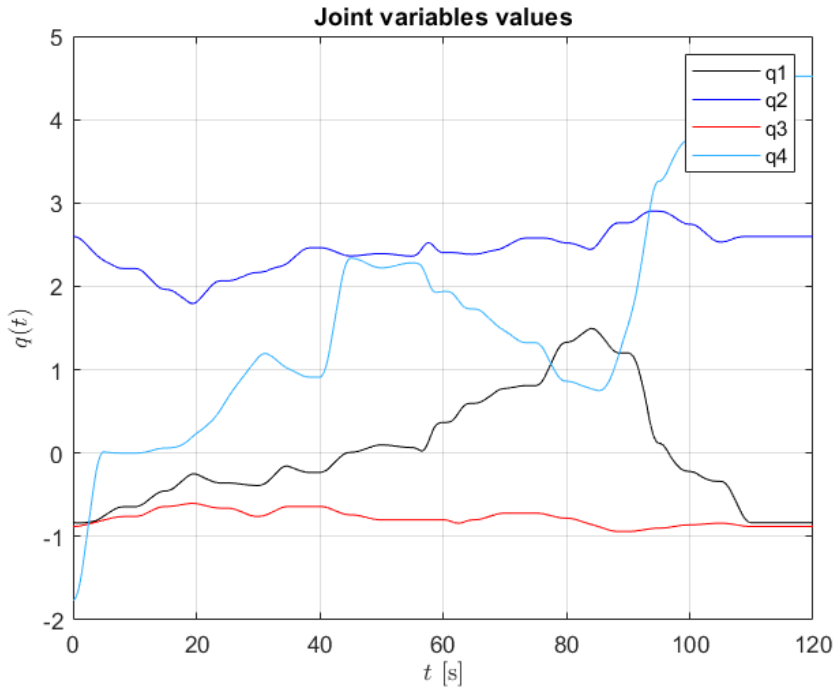


Figure 27: Second order CLIK joint variables values

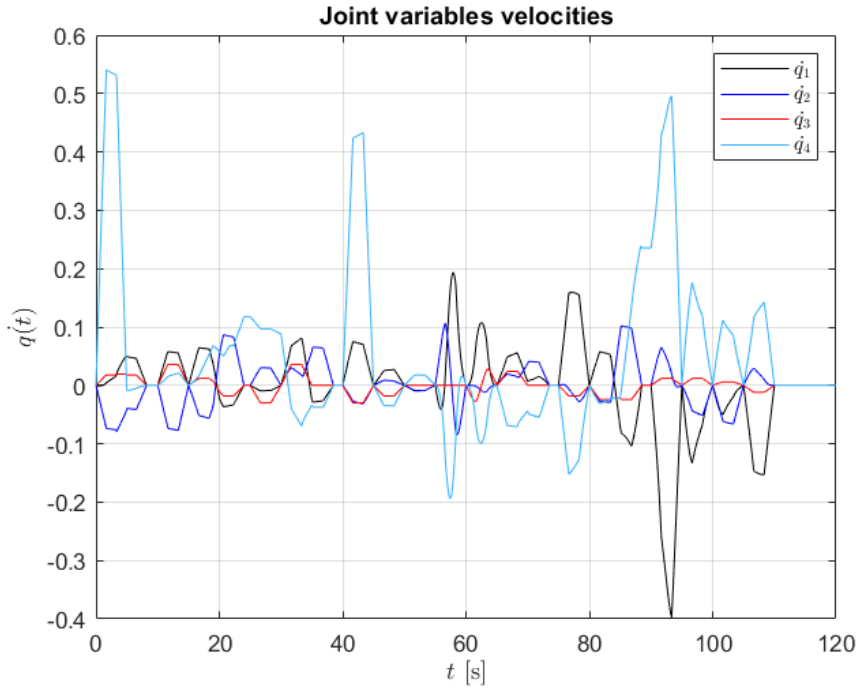


Figure 28: Second order CLIK joint variables velocities

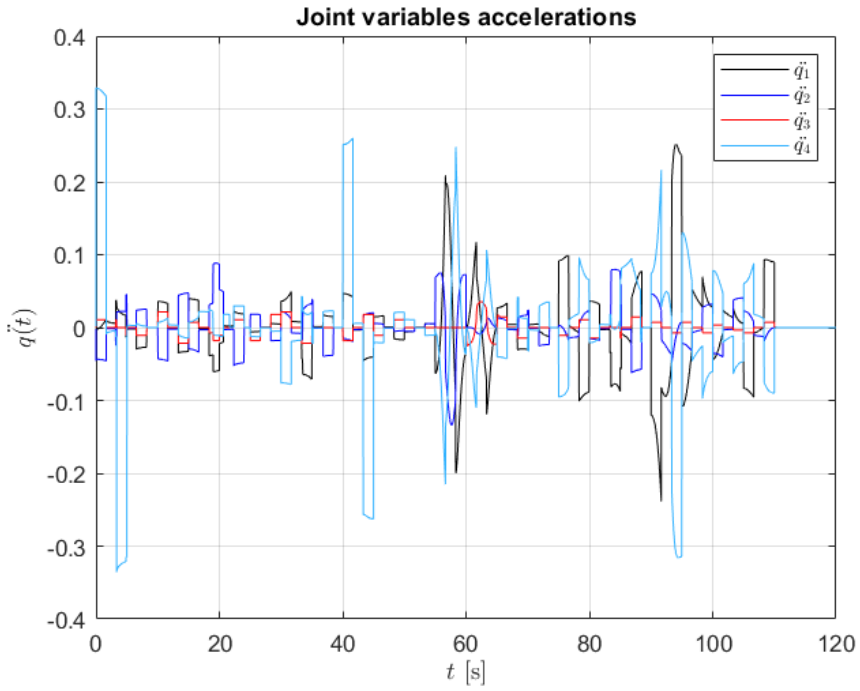


Figure 29: Second order CLIK joint variables accelerations

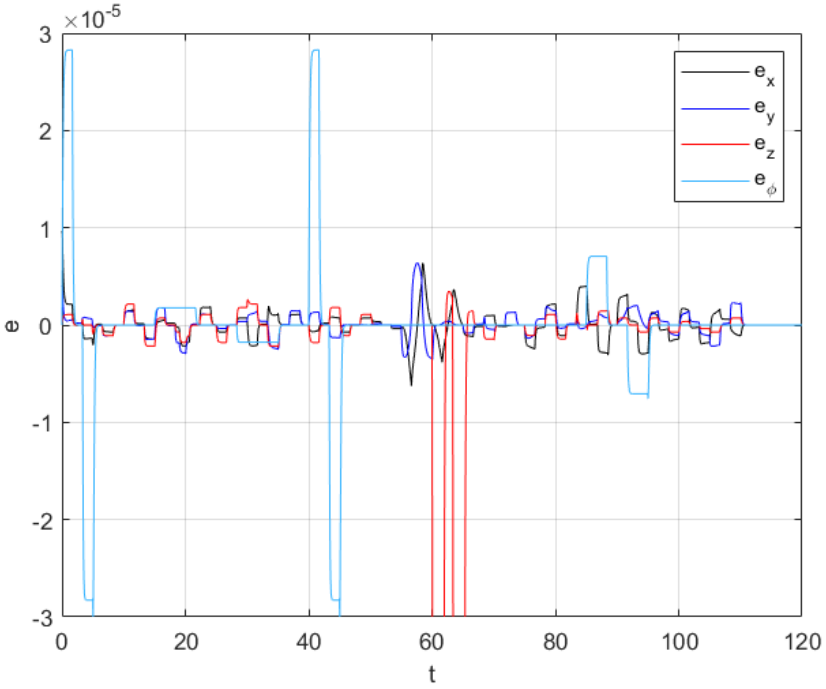


Figure 30: Second order CLIK position tracking error

5 Motion Control Techniques and Dynamic Modeling

This chapter focuses on the application of various techniques for controlling the manipulator's motion. Specifically, the strategies investigated include robust control and adaptive control (executed in joint space), alongside inverse dynamics control (applied in the operational space). The manipulator's core task is to track the trajectory previously defined in Chapter 3, imagining that it starts from the first point of the trajectory.

For robust and adaptive control schemes, the required joint references ($\ddot{q}$) are generated using a second-order CLIK algorithm. Conversely, inverse dynamics control only requires direct operational space references ($x_d, \dot{x}_d, \ddot{x}_d$). All implemented control algorithms rely fundamentally on the precise mathematical model of the manipulator's dynamics.

5.1 Joint-Space Dynamic Model Formulation

The joint-space dynamic model of the manipulator, derived from the Lagrangian formulation, is expressed by the non-linear differential equation:

$$B(q)\ddot{q} + C(q,\dot{q})\dot{q} + F_v\dot{q} + g(q) = \tau \tag{26}$$

Alternatively, the model can be compactly represented by grouping the velocity-dependent and potential energy terms into a single vector, $n(q,\dot{q})$:

$$B(q)\ddot{q} + n(q,\dot{q}) = \tau \tag{27}$$

where $n(q,\dot{q}) = C(q,\dot{q})\dot{q} + F_v\dot{q} + g(q)$.

The constituent terms of Equation 26 are defined as follows:

- $B(q)$: The Inertia Matrix, which is symmetric, positive definite, and configuration-dependent.
- $C(q,\dot{q})$: The matrix representing Coriolis and Centrifugal effects. The chosen factorization uses Christoffel symbols of the first type.
- F_v: The diagonal matrix of Viscous Friction Coefficients.
- $g(q)$: The Gravity Torque Vector acting at the joint axes in the current configuration.
- τ: The vector of Actuation Torques applied at the joints.

5.1.1 Dynamic model analysis

The Inertia Matrix $B(q)$ for the system is a summation over the four links, incorporating both link mass/inertia $(m_{l_i}, I^i_{l_i})$ and motor mass/inertia $(m_{m_i}, I^{m_i}_{m_i})$ contributions:

$$B(q) = \sum_{i=1}^{4} \left(m_{l_i} {J_P^{(l_i)}}^T J_P^{(l_i)} + {J_O^{(l_i)}}^T R_i I^i_{l_i} R_i^T J_O^{(l_i)} + m_{m_i} {J_P^{(m_i)}}^T J_P^{(m_i)} + {J_O^{(m_i)}}^T R_{m_i} I^{m_i}_{m_i} R^T_{m_i} J_O^{(m_i)} \right) \quad (28)$$

The elements of $C(q, \dot{q})$ are derived from the Christoffel symbols of the first type:

$$c_{ij} = \sum_{k=1}^{4} c_{ijk} \dot{q}_k, \quad \text{where} \quad c_{ijk} = \frac{1}{2} \left(\frac{\partial b_{ij}}{\partial q_k} + \frac{\partial b_{ik}}{\partial q_j} - \frac{\partial b_{jk}}{\partial q_i} \right) \quad (29)$$

The $n \times n$ diagonal matrix of viscous friction coefficients is given by the gear reduction ratios (k_{r_i}) and the motor coefficients (F_{m_i}):

$$F_v = \text{diag}[k^2_{r1} F_{m1}, k^2_{r2} F_{m2}, k^2_{r3} F_{m3}, k^2_{r4} F_{m4}] \quad (30)$$

The gravity torque vector $g(q)$ influences primarily the prismatic joint (Joint 3):

$$g(q) = [0, 0, -9.81 m_{l_3}, 0]^T \quad (31)$$

5.2 Dynamic parameters

To fully parametrize the dynamic model, knowledge of the link masses (m_{l_i}), link moments of inertia (I_{l_i}), motor inertias (I_{m_i}), reduction ratios (k_{r_i}), and viscous friction coefficients (F_{m_i}) is required. The specific parameter values adopted for this project are summarized in Table 3:

Link	m_{l_i} **[kg]**	I_{l_i} **[kg·m^2]**	I_{m_i} **[kg·m^2]**	k_{r_i}	F_{m_i} **[Nm·s/rad]**
1	20	4	0.01	1	0.00005
2	20	4	0.01	1	0.00005
3	10	0	0.005	50 rad/m	0.01
4	0	1	0.001	20	0.005

Table 3: Dynamic Parameters of the SCARA Manipulator

5.3 Robust Control

Robust control is a methodology that applies a feedback linearization technique to the non-linear terms of the dynamic model, specifically engineered to operate under the premise of non-perfect compensation of these terms.

The control input, τ, is chosen to compensate for the estimated dynamic effects, $\hat{B}(q)$ and $\hat{n}(q, \dot{q})$, amplified by an auxiliary input y:

$$\tau = \hat{B}(q)y + \hat{n}(q, \dot{q}) \tag{32}$$

Here, $\hat{B}(q)$ and $\hat{n}(q, \dot{q})$ represent the compensated terms of the dynamic model. The auxiliary input y incorporates feedforward acceleration, feedback stabilization, and a robustness term w:

$$y = \ddot{q}_d + K_D\dot{\tilde{q}} + K_P\tilde{q} + w \tag{33}$$

Where the components are defined as:

- $\tilde{q}$: The vector of the position error at the joints.
- $\dot{\tilde{q}}$: The vector of the velocity error at the joints.
- $\ddot{q}_d$: The acceleration vector of reference for the joints.
- K_P and K_D: The positive-definite control gain matrices.
- w: The control term chosen according to a sliding mode (versor) law, designed to enforce robustness against uncertainties.

The joint position and velocity errors are combined into the state error vector ξ:

$$\xi = \begin{pmatrix} \tilde{q} \\ \dot{\tilde{q}} \end{pmatrix}$$

5.3.1 Sliding Mode Control and Chattering Mitigation

The robustness term w is designed to force the system dynamics towards the desired sliding subspace, $z = D^T Q \xi = 0$. By enforcing this subspace's dynamics, the system is driven towards $\xi = 0$, guaranteeing asymptotic convergence of both position and velocity errors.

The constant dimension matrix $D \in \mathbb{R}^{8\times 4}$ is defined as:

$$D = \begin{pmatrix} 0_4 \\ I_4 \end{pmatrix}$$

and the positive definite matrix Q for the simulation was chosen as:

$$Q = \begin{pmatrix} 10I_4 & 1000I_4 \\ 1000I_4 & 10I_4 \end{pmatrix}$$

The control law w employs a saturation boundary, ϵ, to manage the dynamics around the sliding subspace:

$$w = \begin{cases} \frac{\rho}{\|z\|} z, & \text{for } \|z\| \geq \epsilon \\ \frac{\rho}{\epsilon} z, & \text{for } \|z\| < \epsilon \end{cases} \tag{34}$$

where ρ is the control law gain. The boundary factor ϵ is a necessary saturation term; without it, the control law would become singular ($\|z\| = 0$) once the system reaches the sliding surface, leading to infinite control action.

The presence of the non-zero saturation boundary ϵ causes continuous high-frequency oscillation of the system error dynamics around the sliding subspace. This phenomenon is known as chattering , which consists of continuos oscillations in the control action τ and the controlled quantities.

For the primary simulation, the parameters were set as $\rho = 3$ and $\epsilon = 0.3$. An additional simulation was conducted using a larger value, $\epsilon = 0.5$, to specifically demonstrate system behavior that successfully avoids the chattering phenomenon.

The Simulink scheme which implements the robust control technique is the following:

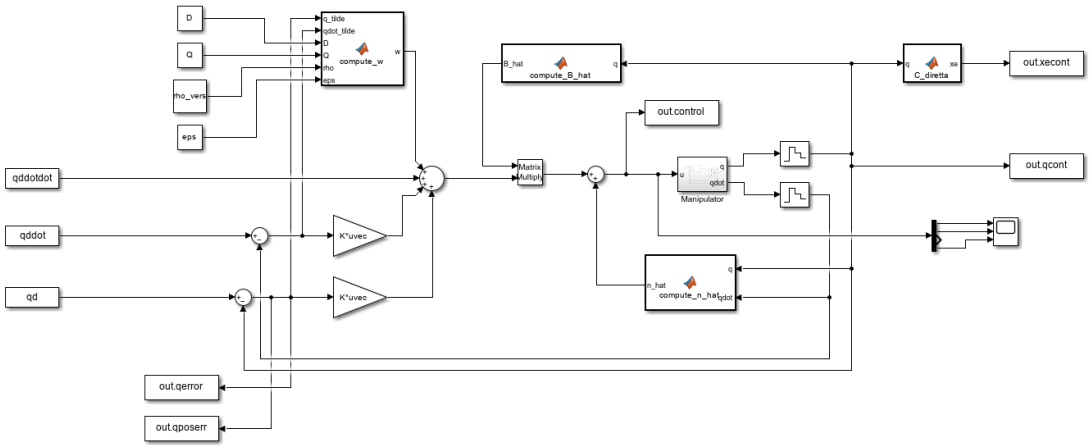


Figure 31: Robust control Simulink scheme

The results of the simulation with $\epsilon = 0.01$ are shown here:

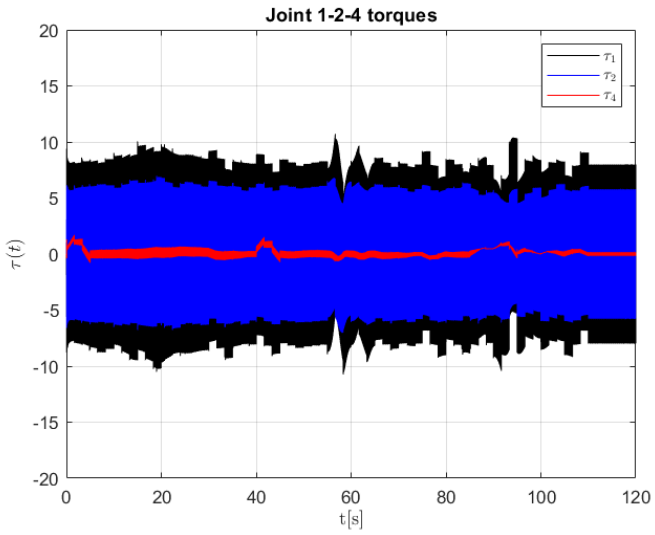


Figure 32: Joint torques 1-2-4 with $\epsilon = 0.01$

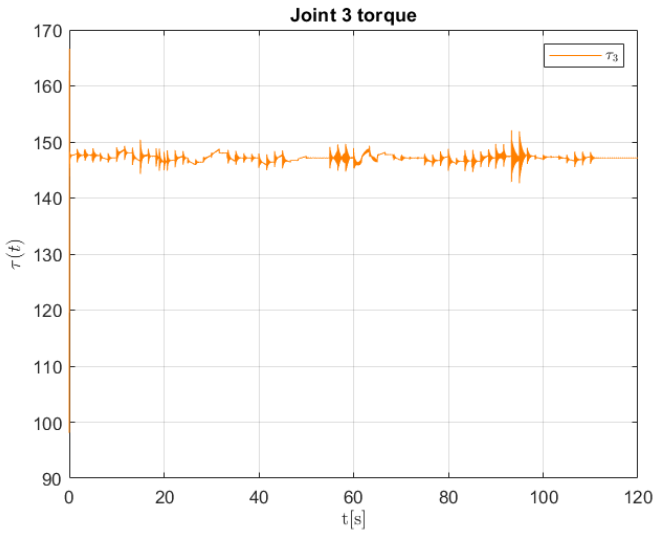


Figure 33: Joint 3 torque with $\epsilon = 0.01$

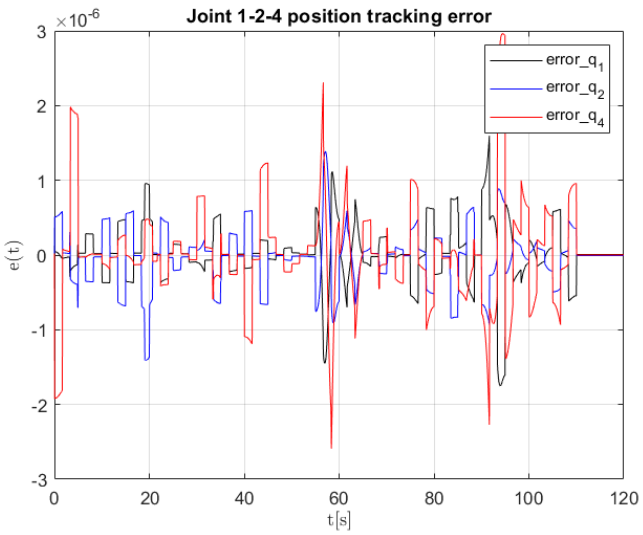


Figure 34: Position tracking error with $\epsilon = 0.01$ of joints 1-2-4

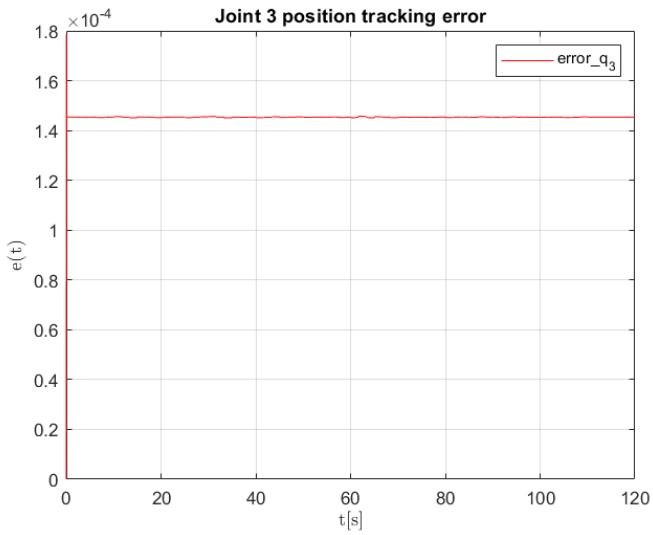


Figure 35: Position tracking error with $\epsilon = 0.01$ of joint 3

By increasing the ϵ factor to 0.5, we can appreciate how the behaviour of joint torques changes:

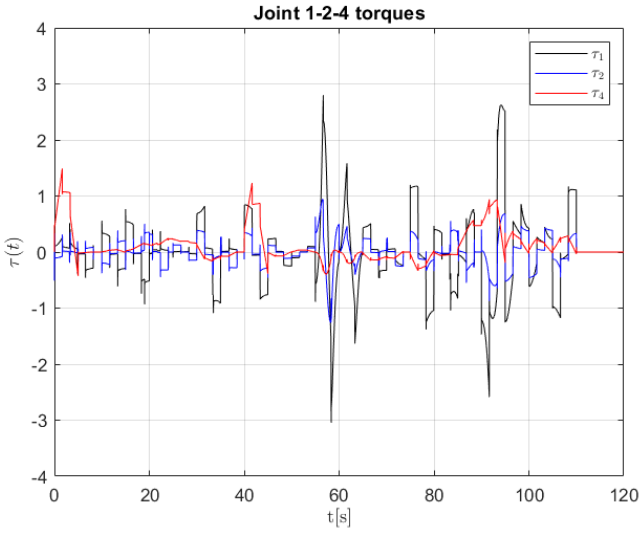


Figure 36: Joint torques 1-2-4 with $\epsilon = 0.3$

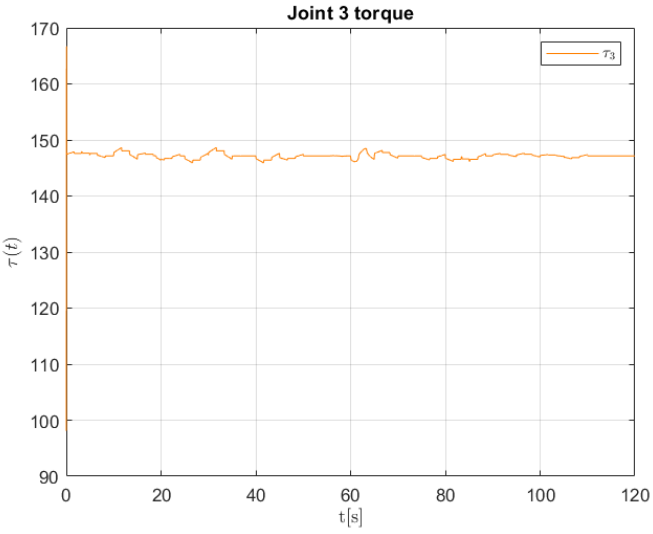


Figure 37: Joint 3 torque with $\epsilon = 0.3$

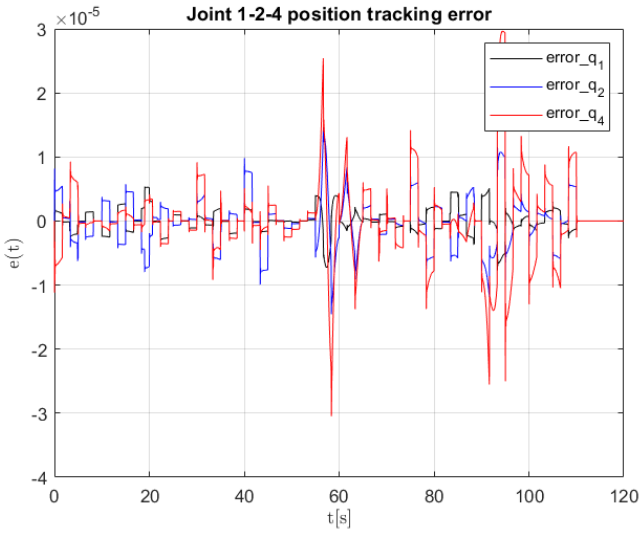


Figure 38: Position tracking error of joints 1-2-4 with $\epsilon = 0.3$

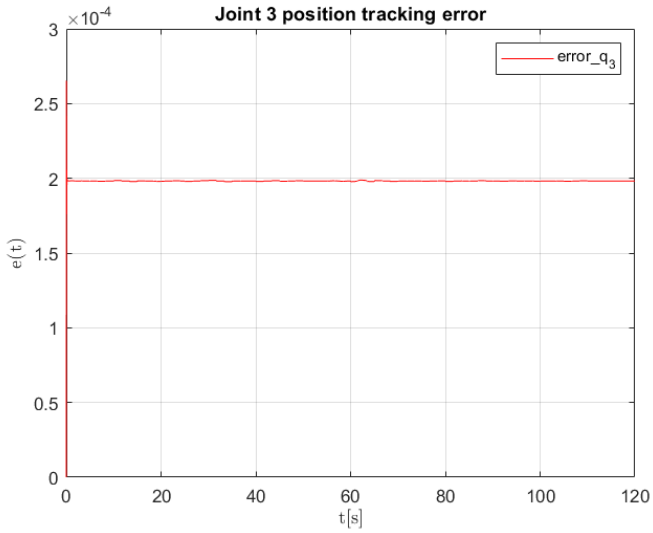


Figure 39: Position tracking error of joint 3 with $\epsilon = 0.3$

Those simulation results highlight the importance of the choice of ϵ as it deeply influences the oscillatory behaviour of the control input and the amplitude of the position tracking error.

5.4 Adaptive Control

Adaptive control represents a robust strategy designed to manage systems where dynamic parameters may either vary over time or be initially unknown. This method dynamically adjusts the controller parameters in real-time to accommodate system uncertainties and preserve overall asymptotic stability. The effectiveness of this approach stems from the inherent property of the manipulator's dynamic model being linear with respect to a vector of constant dynamic parameters, π.

The dynamic model can thus be expressed in the linear parametric form:

$$B(q)\ddot{q} + C(q,\dot{q})\dot{q} + F_v\dot{q} + g(q) = Y(q,\dot{q},\ddot{q})\pi \tag{35}$$

where $Y \in \mathbb{R}^{4\times p}$ is the Regressor Matrix, which depends on joint positions, velocities, and accelerations, and p is the number of dynamic parameters.

In the case study considered here, a vector of 15 dynamic parameters was chosen:

$$\pi = [m_{l_1}, I_{l_1}, I_{m_1}, F_{m_1}, m_{l_2}, I_{l_2}, I_{m_2}, F_{m_2}, m_{l_3}, I_{m_3}, F_{m_3}, m_{l_4}, I_{l_4}, I_{m_4}, F_{m_4}]^T$$

Assuming the use of parameter estimates ($\hat{\pi}$) in place of the true values, the adaptive control law is chosen as:

$$u = \hat{B}(q)\ddot{q}_r + \hat{C}(q,\dot{q})\dot{q}_r + \hat{F}_v\dot{q}_r + \hat{g}(q) + K_D\sigma = Y(q,\dot{q},\dot{q}_r,\ddot{q}_r)\hat{\pi} + K_D\sigma \tag{36}$$

The stability of the adaptive scheme is guaranteed by an adaptation law that computes the rate of parameter adjustment, $\dot{\hat{\pi}}$, based on the generalized error σ:

$$\dot{\hat{\pi}} = K_{\pi}^{-1} Y^{T}(q, \dot{q}, \dot{q}_r, \ddot{q}_r)\sigma \tag{37}$$

The essential intermediate terms are defined as follows:

- $\sigma = \dot{q}_r - \dot{q}$: The generalized velocity error.
- $\dot{q}_r = \dot{q}_d + \Lambda\tilde{q}$: The reference velocity, representing a linear combination of desired velocity ($\dot{q}_d$) and position error ($\tilde{q} = q_d - q$).
- $\ddot{q}_r = \ddot{q}_d + \Lambda\dot{\tilde{q}}$: The reference acceleration, derived from desired acceleration ($\ddot{q}_d$) and velocity error ($\dot{\tilde{q}}$).
- Λ and K_D are positive-definite diagonal gain matrices.
- K_{π}: The adaptation gain matrix. The diagonal value of K_{π} for the i-th parameter indicates the assumed accuracy of that parameter: a higher value requires a minor correction, while a lower value (lighter weight) directs the algorithm to focus its estimation effort on that specific uncertain parameter.

Assuming high prior knowledge for most parameters, a heavier weight is assigned to all terms in K_{π} except for the one corresponding to m_{l_4}, enabling the algorithm to focus robustly on estimating the mass of the last link and applying marked corrections to it.

The Simulink scheme used for implementing adaptive control is the following:

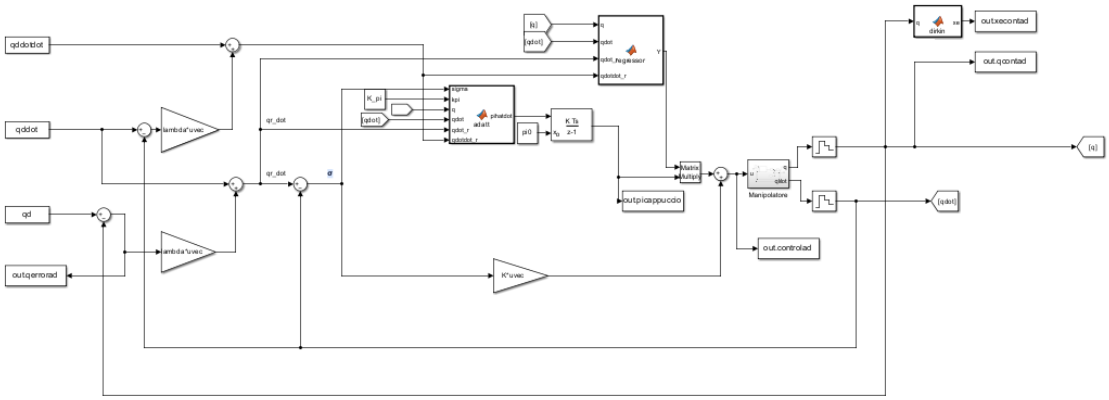


Figure 40: Adaptive control Simulink scheme

Hereafter, simulation results are presented:

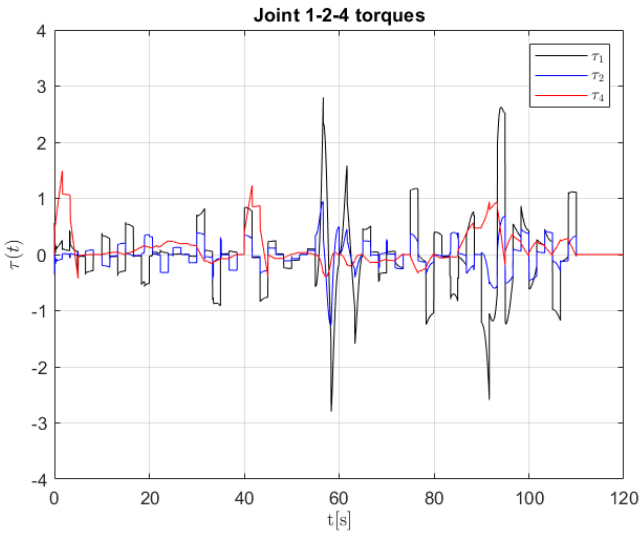


Figure 41: Joint 1-2-4 torques with adaptive control

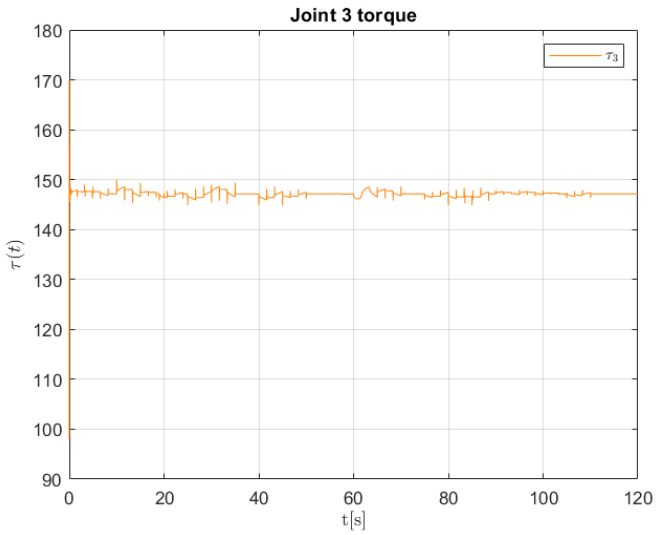


Figure 42: Joint 1-2-4 torques with adaptive control

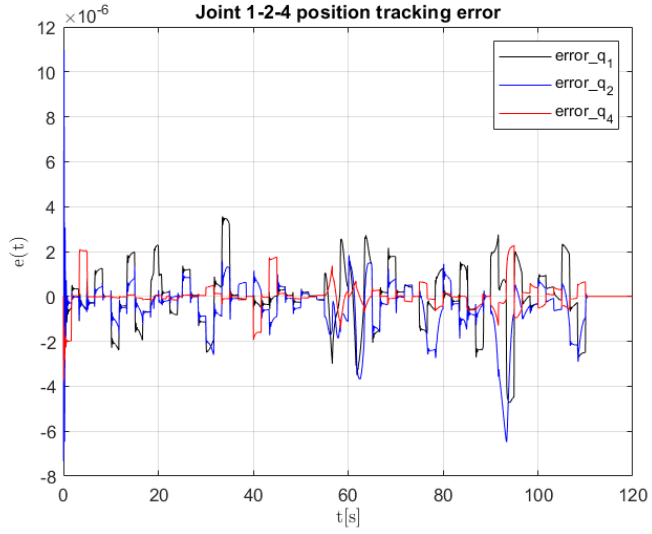


Figure 43: Position tracking error of joints 1-2-4

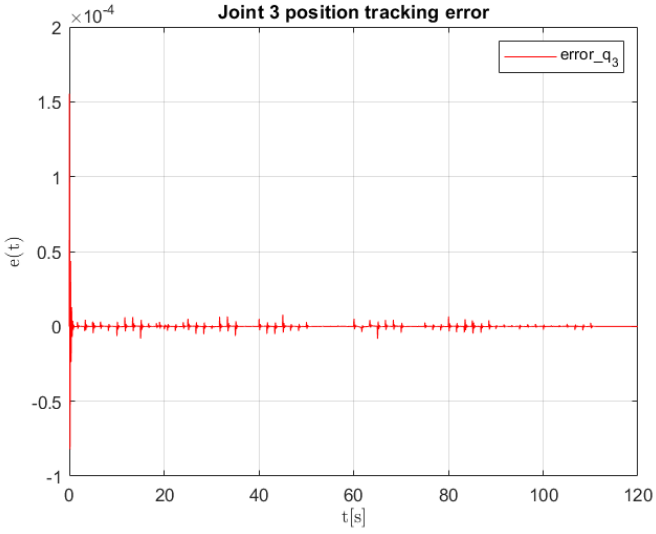


Figure 44: Position tracking error of joint 3

5.5 Inverse Dynamic Control in Operational Space

Motion specifications are commonly defined in the operational space. While conventional methods utilize inverse kinematics to translate these references into joint-space inputs, an alternative and direct approach is offered by the Inverse Dynamic Control (IDC) scheme, which operates entirely within the operational space.

The IDC methodology leverages feedback linearization by selecting a control input τ that compensates for the known non-linear dynamics:

$$\tau = B(q)y + n(q, \dot{q}) \tag{38}$$

By substituting this control law into the manipulator's dynamic model, $B(q)\ddot{q} + n(q, \dot{q}) = \tau$, the complex non-linear system is decoupled and simplified to a linear double integrator in the joint space:

$$\ddot{q} = y$$

To impose the desired dynamics at the end-effector level, we utilize the relationship between operational space acceleration, $\ddot{x}_e$, and joint acceleration, $\ddot{q}$:

$$\ddot{x}_e = J_A(q)\ddot{q} + \dot{J}_A(q, \dot{q})\dot{q}$$

For a non-redundant manipulator, the auxiliary input y is strategically chosen to achieve cancellation of kinematic coupling and to enforce stable error dynamics:

$$y = J_A^{-1}(q)\left(\ddot{x}_d + K_D\dot{\tilde{x}} + K_P\tilde{x} - \dot{J}_A(q, \dot{q})\dot{q}\right) \tag{39}$$

Where the feedback terms $\tilde{x} = x_d - x_e$ and $\dot{\tilde{x}}$ represent the position and velocity errors in the operational space, and K_P and K_D are the proportional and derivative gain matrices, respectively.

Substituting the control law y (Equation 39) into the acceleration dynamics results in the linear, second-order closed-loop error dynamics:

$$\ddot{\tilde{x}} + K_D\dot{\tilde{x}} + K_P\tilde{x} = 0$$

By imposing K_P and K_D as positive-definite diagonal matrices, this system is guaranteed to be globally asymptotically stable, ensuring that the end-effector accurately and exponentially converges to the desired trajectory.

The Simulink control scheme is the follwing:

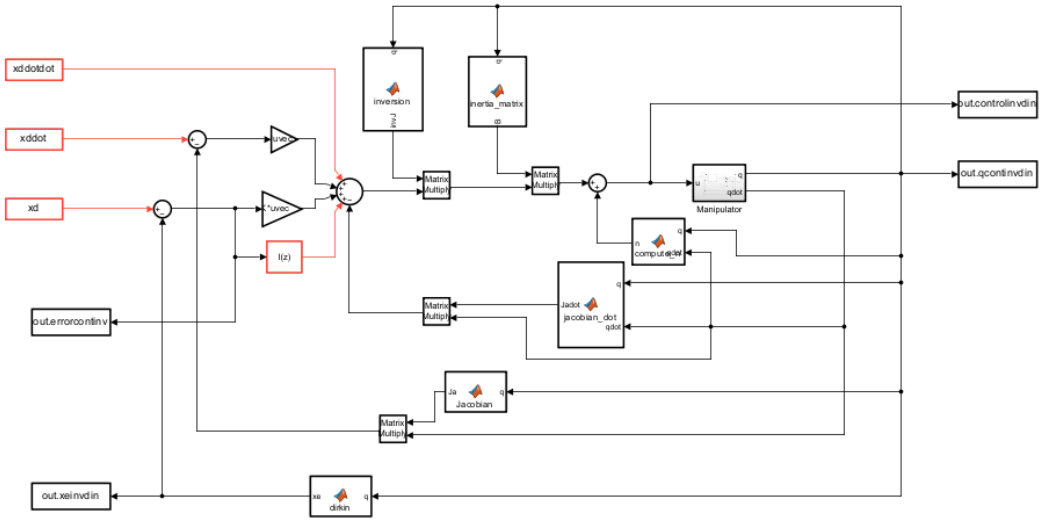


Figure 45: Operational space inverse dynamics control Simulink scheme

Simulation results are the following:

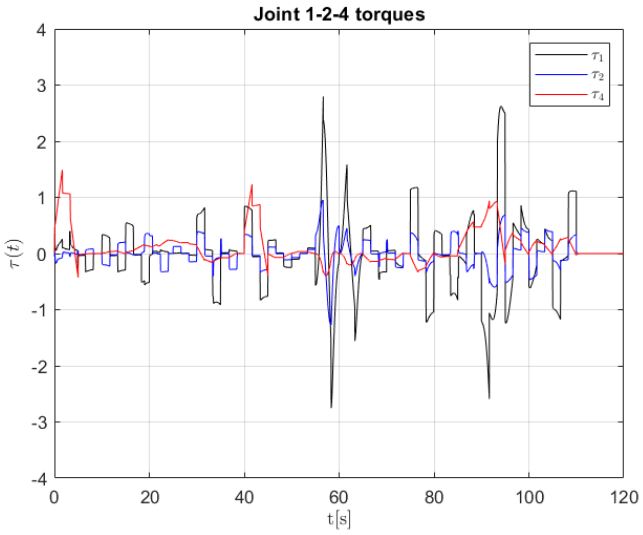


Figure 46: Joints 1-2-4 torques

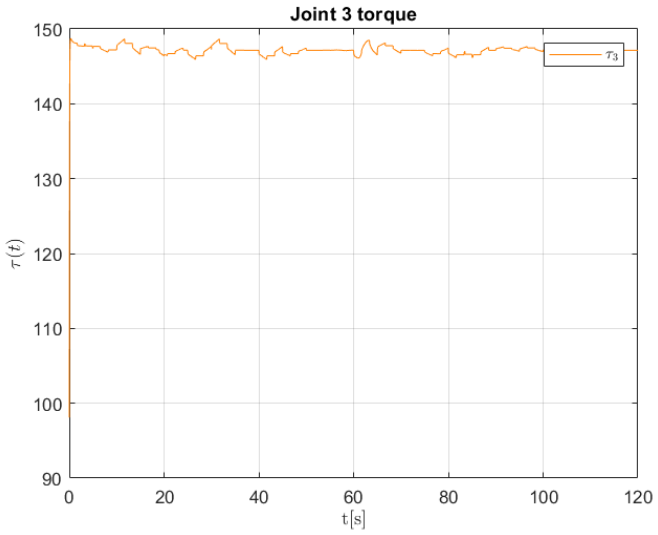


Figure 47: Joint 3 torque

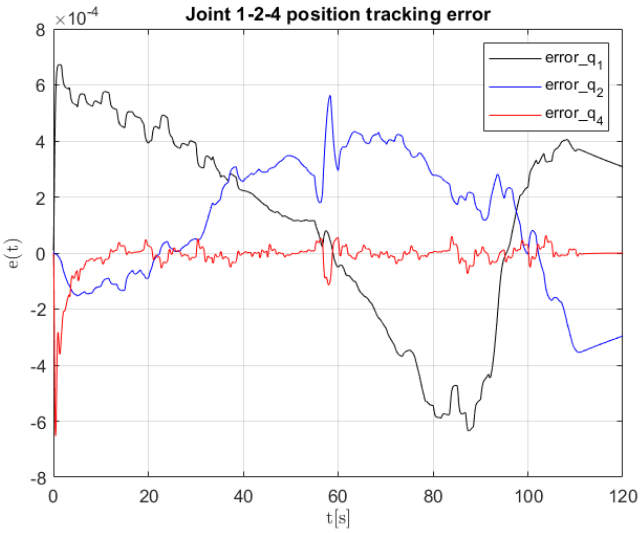


Figure 48: Position tracking error of joints 1-2-4

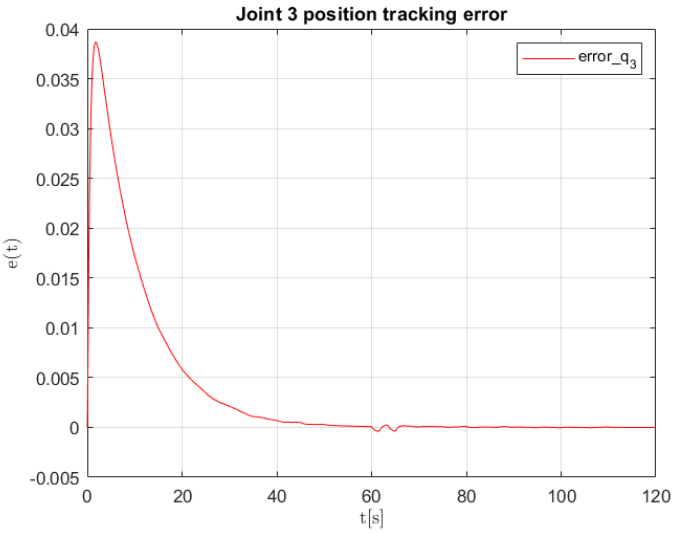


Figure 49: Position tracking error of joint 3